

Performance Testing

Date	02 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (functional/API checks) and JMeter (load testing)
Type of Testing	Load Testing and Stress Testing
Target Module / API	/predict endpoint (Credit Card Approval Prediction API)
Test Environment	Cloud / Hosted Server (Flask app deployed with IBM Watson ML)
Test Date	29 June 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single applicant submits prediction request	10	60	Response returned within 2 seconds
2	Multiple applicants submit requests simultaneously	50	120	System handles load without errors
3	Peak load / stress test on prediction endpoint	150	180	System remains stable, error rate < 1%
4	Sustained load over an extended period	75	300	Consistent response time, no memory leaks

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Well within target
2	Response Time (Max)	< 5 seconds	3.8 seconds	Pass	Occasional spike under peak load
3	Throughput (Req/sec)	-	42 req/sec	Pass	Stable under sustained load

4	Error Rate	< 1%	0.3%	Pass	Minor timeouts under stress test
5	CPU Utilization	< 80%	68%	Pass	Peaked during stress test
6	Memory Utilization	< 80%	71%	Pass	Stable, no memory leaks observed

Step 4: Observations & Analysis

Key Findings:

The system consistently met response time targets under normal load, with average response times around 1.4 seconds. Throughput remained stable at approximately 42 requests per second.

Bottlenecks Identified:

Slight increase in response time and CPU usage observed during peak/stress testing with 150 concurrent virtual users. No major bottlenecks were found under normal load conditions.

Optimization Steps Taken:

Enabled model caching to avoid reloading the trained model on every request, and optimized data preprocessing steps to reduce per-request latency.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Postman output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]